

Lesson 3.2 – The Twelve Patterns

Lesson 3.2 – The Twelve Patterns

Twelve prompts cover ninety percent of real engineering work with AI. This lesson is the library. Memorize the shapes. Adapt the words.

By the end of this lesson:

- You will have twelve copy-pasteable prompt templates organized by intent.
- Each pattern names the verb, the situation it fits, and the senior-grade variant.
- You will have an interview demo prompt for each pattern – the one to walk through if asked.

Treat this lesson as a reference. Read once. Bookmark. Return often.

1. How to use the library

Each pattern has the same shape:

- Name – what to call it in your head.
- When you reach for it – the situation.
- Template – fill-in-the-blanks structure.
- Real example – adapted to a real codebase decision.
- Interview soundbite – what you say when asked about this pattern.

Memorize the names. The names alone make you sound deliberate.

2. Pattern 1 — The Project-Shaping Prompt

When you reach for it

You are about to start a new project. Before any code, you want to confirm the model understands the problem.

Template

"I want to build a [DOMAIN] app. The data is [DATA SHAPE IN ONE PARAGRAPH]. The user can [3 TO 5 USER ACTIONS]. Do not write code yet. Confirm you understand the data and the actions, propose a tab UI, and call out anything ambiguous."

Real example

"I want to build a fantasy snooker league app. The data is eight teams, each picking match winners across five rounds; matches have winners or are unplayed; scoring is 3 for a correct pick, 1 for wrong, null for unplayed. The user can see standings, drill into one team's picks, see analytics charts, and switch rounds. Do not write code yet. Confirm understanding, propose a five-tab UI, and ask any clarifying questions."

Interview soundbite

"My first prompt on a new project is always a no-code prompt. I tell the model what I think the problem is and ask it to push back. Forces ambiguities to surface in 30 seconds instead of after I have shipped the wrong thing."

3. Pattern 2 — The Data-Modeling Prompt

When you reach for it

You know the entities. You want strict types before any UI exists.

Template

“Define TypeScript interfaces for [ENTITIES]. Strict mode — no any, no implicit optional fields. For each interface, comment any non-obvious invariant. Use composition rather than inheritance when interfaces share fields.”

Real example

“Define TypeScript interfaces for Match, Team, PlayerInfo, ScoreDetail, TeamScores. Strict — no any. Match.winner is optional only when the match has not been played. Team.r1, r2, qf, sf are arrays where index i corresponds to round-N match index i — comment that invariant explicitly.”

Interview soundbite

“Strict types up front are the cheapest tax I can pay. They make every subsequent prompt easier because the model has shapes to refer to.”

4. Pattern 3 — The Pure-Logic Prompt

When you reach for it

You want testable, side-effect-free functions before any React.

Template

“Write [N] pure functions that take [INPUTS] and return [OUTPUTS]. No side effects. No I/O. No framework imports. Add JSDoc comments explaining each. Output as a single .ts file.”

Real example

“Write two pure functions: scorePick(pick: string, match: Match): number | null and calculateTeamScores(team: Team): TeamScores. scorePick returns 3 if pick equals match.winner, 1 if pick is wrong, null if match has no winner. calculateTeamScores aggregates across all five rounds. No I/O. No React. Output as lib/scoring.ts.”

Interview soundbite

“I write the pure logic before any UI. It is the part of the codebase that survives every refactor, and it is trivially testable. I will not start UI work until the logic is solid.”

5. Pattern 4 – The First-Component Prompt

When you reach for it

You have data and types. You want the smallest possible thing on screen to prove the data is real.

Template

“Render the simplest possible visualization of [DATA]. Use a functional React component with explicit props typed from [TYPE FILE]. No useState. No useMemo. Inline styles. Do not extract sub-components yet.”

Real example

“Render a basic StandingsTab from an array of TeamWithScores. Use a plain HTML table. No useState, no useMemo, no extracted sub-components. Show team name and total. Inline styles only – match the green/yellow palette in lib/constants.ts.”

Interview soundbite

“I always start with the smallest thing that proves the data is real. Inline styles. No abstractions. Then I let abstractions earn their way in.”

6. Pattern 5 — The Progressive-Enhancement Prompt

When you reach for it

You have a working component. You want to add one feature without redesigning anything.

Template

“Add [ONE FEATURE] to [EXISTING COMPONENT at FILEPATH]. Do not redesign anything. Do not refactor unrelated code. If the change touches more than three files, stop and tell me which files first. Return a unified diff.”

Real example

“Add a per-round column to the standings table in components/tabs/StandingsTab.tsx. Show R1 / R2 / QF / SF / Final scores. Do not redesign the table. Do not extract sub-components. If this touches more than three files, stop and tell me. Diff format.”

Interview soundbite

“This is the most useful prompt in my library. The ‘do not redesign’ clause and the ‘stop if more than three files’ clause are scar tissue from the times the AI helpfully rewrote my whole module.”

7. Pattern 6 — The Refactor Prompt

When you reach for it

You want a structural change with no behavior change.

Template

“Refactor [CURRENT FILE] into an [TARGET PATTERN]. Move only [WHAT MOVES]. Do not change behavior. Do not change the public API of [PUBLIC SURFACE]. Keep all styling identical. Add no new dependencies.”

Real example

“Refactor components/SnookerFantasyLeague.tsx into an orchestrator pattern. The orchestrator owns activeTab and selectedTeam with useState. Tab components receive props. Do not change behavior. Do not change styling. Tests must continue to pass.”

Interview soundbite

“Refactors are the prompts where I am strictest about constraints. Behavior must not change. Styling must not change. Public API must not change. Without those clauses, the AI helpfully ‘improves’ things and the diff becomes unreviewable.”

8. Pattern 7 — The Debug Prompt

When you reach for it

A test is failing or a behavior is wrong. You want a hypothesis-driven fix.

Template

“Bug: [SPECIFIC OBSERVED BEHAVIOR]. Expected: [SPECIFIC EXPECTED BEHAVIOR]. Reproducer: [STEPS]. Hypothesis: [YOUR GUESS]. Confirm or refute the hypothesis. Then patch the smallest part of the code that fixes it. Do not change tests.”

Real example

“Bug: scorePick returns 1 for unplayed matches. Expected: it should return null. Reproducer: scorePick(‘Williams’, { p1: ‘Williams’, p2: ‘Wakelin’ }). Hypothesis: the function checks ‘pick === winner’ before checking ‘has winner’, and falls through to the 1-return. Confirm or refute. Patch only lib/scoring.ts. Do not modify tests.”

Interview soundbite

“I always include my own hypothesis when debugging with AI. Even a wrong hypothesis is useful — it gives the model something to confirm or refute. Without it, the model guesses, and guessing is what got me into the bug.”

9. Pattern 8 — The Test Prompt

When you reach for it

You have working code that needs coverage.

Template

“Write [N] [unit / integration / E2E] tests for [TARGET]. Cover these specific cases: [LIST]. Use [TEST FRAMEWORK]. Mirror the style of [EXISTING TEST FILE if any]. One assertion per test.”

Real example

“Write three Vitest unit tests for scorePick in lib/scoring.ts. Cases: correct pick returns 3; wrong pick returns 1; unplayed match returns null. Mirror the style of lib/utls.test.ts. One assertion per test. Use describe/it. Do not test calculateTeamScores in this turn.”

Interview soundbite

“Tests are where I am most explicit about cases. The model will happily generate twenty tests of weak quality. I prefer three tests that hit the boundary cases I named.”

10. Pattern 9 — The Design / Architecture Prompt

When you reach for it

A decision needs to be made. You want options and trade-offs without a default pick.

Template

"I need to decide [DECISION]. Constraints: [LIST]. Constraints I am not willing to negotiate: [LIST]. List three approaches with their trade-offs. Do not pick one. Surface any constraints I missed."

Real example

"I need to decide where to cache: CDN, application server, or browser. Constraints: total monthly cost cap is \$X, app servers have spare CPU and tight memory, 30% of users are on flaky networks. Constraints I am not willing to negotiate: no third-party CDN beyond our existing Cloudfront. List three approaches with trade-offs. Do not pick one. Surface anything I missed."

Interview soundbite

"When I am genuinely undecided, I tell the model not to pick. The 'list three approaches' framing forces it to surface options I would have missed. The choice still belongs to me."

11. Pattern 10 – The Documentation Prompt

When you reach for it

You have working code that needs explanation for humans.

Template

"Generate [DOC TYPE] for [TARGET]. Audience: [WHO]. Tone: [STYLE]. Length: [LIMIT]. Cover [SECTIONS]. Do not invent features. If anything is unclear from the code, list questions instead of guessing."

Real example

"Generate a README for the lib/scoring.ts module. Audience: a junior engineer joining next month. Tone: technical but friendly. Length: under 300 words. Cover the two functions, the invariant about array index correspondence, and

one example call each. Do not invent features. List anything unclear instead of guessing.”

Interview soundbite

“The ‘do not invent features’ clause is critical. AI will happily document behavior the code does not have. The ‘list questions instead’ clause turns documentation into a sanity check on the code itself.”

12. Pattern 11 – The Prompt-Improver Prompt

When you reach for it

A prompt of yours produced bad output. You want a better prompt.

Template

“Here is the prompt I wrote: [PROMPT]. Here is the output it produced: [OUTPUT]. Here is what I actually wanted: [DESIRED]. Suggest a stronger prompt that closes the gap. Explain which of the six prompt components it strengthens.”

Real example

“Prompt: ‘add a loading state to my button.’ Output: a new component called Loader at the wrong path with new dependencies. Wanted: a small change to PrimaryButton that adds isLoading prop. Suggest a stronger prompt and explain which components it strengthens.”

Interview soundbite

“When a prompt produces bad output, the prompt is the bug, not the model. I have a meta-prompt that asks the AI to critique my prompt and suggest a better one. It works almost every time.”

13. Pattern 12 – The Post-Mortem Prompt

When you reach for it

A bug shipped. You want a structured retro.

Template

“We shipped a bug: [DESCRIBE]. Customer impact: [DESCRIBE]. Root cause: [WHAT YOU FOUND]. Detection: [HOW WE FOUND IT]. Remediation: [WHAT WE DID]. Generate a postmortem document with Summary, Timeline, Root Cause, Contributing Factors, What Went Well, What Could Improve, Action Items. Be honest about what we missed. No blame language.”

Real example

“We shipped a payments bug: partial-charge edge case mishandled. Impact: 4 hours, ~50 affected accounts. Root cause: missing null check after gateway response. Detection: customer support ticket, then alerting fired. Remediation: hotfix + refund script. Generate a full postmortem doc. Honest about what we missed. No blame.”

Interview soundbite

“Postmortems are one of the highest-value places to use AI. The structure is well-known, the tone is hard to nail, and the model is great at both. I always edit the draft, but the draft saves me an hour.”

14. The pattern lookup table

When the interviewer presents a hypothetical, you reach for the right pattern in seconds.

If the situation is...	Reach for pattern
Brand new project	1 (Project-Shaping)
Need types	2 (Data-Modeling)
Need pure functions	3 (Pure-Logic)

If the situation is...	Reach for pattern
Smallest UI	4 (First-Component)
Add one feature	5 (Progressive Enhancement)
Restructure without behavior change	6 (Refactor)
Bug	7 (Debug)
Coverage	8 (Test)
Decision	9 (Design / Architecture)
Doc	10 (Documentation)
AI gave bad output	11 (Prompt Improver)
Incident retro	12 (Post-Mortem)

Memorize the table. In an interview, when asked “how would you tackle X?”, lead with: “I would start with a [PATTERN NAME] prompt...”

15. The senior soundbite — naming patterns out loud

“I keep about a dozen prompt patterns in my head — project-shaping, data-modeling, pure-logic, refactor, debug, test, design, doc, post-mortem. Naming them is how I avoid one-off ad-hoc prompts. The pattern dictates the verb, the constraints, and the output format. The unique part of any prompt is the deliverable and the context.”

That paragraph signals you have done this for long enough to abstract over it. **Drop it once per interview.**

16. Common pitfalls

16.1. Wrong pattern for the job

Using the Refactor template when you really need a Design template — you got code when you wanted options.

16.2. Stuffing two patterns into one prompt

“Refactor X and add tests and update the docs” — three patterns crammed into one. Output is mediocre across all three. Run them as three prompts.

16.3. Skipping the pattern step

Prompting from a blank cursor instead of starting from a template. Slower, weaker output. **Even five seconds of “which pattern is this?” improves the prompt.**

17. Practice

Pick three real tasks from your last week. For each, identify the pattern that fits, fill in the template, and run the prompt. Compare to whatever you would have written off the cuff.

By the end of the week, the templates will feel automatic. **Patterns become invisible once internalized.**

18. CHECK YOURSELF

- ☐ Can you list the 12 patterns by name without looking?
 - ☐ Can you fill in any one template from memory?
 - ☐ Do you know which pattern fits these situations: bug, new project, refactor, doc, decision?
 - ☐ Have you delivered the section-15 senior soundbite out loud at least once?
-

19. Where you are now

You have a library of twelve patterns plus the discipline to deploy them. Move on to **03-rejecting-ai-output.md** — the part most engineers do worst, and the part interviewers most want to see.